

SOURCE LOG: src\chroma_core\tensor_math.py

```
import torch
from einops import rearrange
from torch import Tensor

try:
    from flash_attn.flash_attn_interface import flash_attn_func
    _HAS_FLASH = True
except ImportError:
    _HAS_FLASH = False

def rope(pos: Tensor, dim: int, theta: int) -> Tensor:
    assert dim % 2 == 0
    scale = torch.arange(0, dim, 2, dtype=torch.float64, device=pos.device) / dim
    omega = 1.0 / (theta ** scale)
    out = torch.einsum("...n,d->...nd", pos, omega)
    out = torch.stack([torch.cos(out), -torch.sin(out), torch.sin(out), torch.cos(out)], dim=-1)
    out = rearrange(out, "b n d (i j) -> b n d i j", i=2, j=2)
    return out.float()

def apply_rope(xq: Tensor, xk: Tensor, freqs_cis: Tensor) -> tuple[Tensor, Tensor]:
    xq_ = xq.float().reshape(*xq.shape[:-1], -1, 1, 2)
    xk_ = xk.float().reshape(*xk.shape[:-1], -1, 1, 2)
    xq_out = freqs_cis[..., 0] * xq_[..., 0] + freqs_cis[..., 1] * xq_[..., 1]
    xk_out = freqs_cis[..., 0] * xk_[..., 0] + freqs_cis[..., 1] * xk_[..., 1]
    return xq_out.reshape(*xq.shape).type_as(xq), xk_out.reshape(*xk.shape).type_as(xk)

def attention(q: Tensor, k: Tensor, v: Tensor, pe: Tensor, mask: Tensor) -> Tensor:
    q, k = apply_rope(q, k, pe)
    if _HAS_FLASH and mask is None and q.is_cuda:
        x = flash_attn_func(
            rearrange(q, "B H L D -> B L H D").contiguous(),
            rearrange(k, "B H L D -> B L H D").contiguous(),
            rearrange(v, "B H L D -> B L H D").contiguous(),
            dropout_p=0.0, softmax_scale=None, causal=False,
        )
        x = rearrange(x, "B L H D -> B H L D")
    else:
        x = torch.nn.functional.scaled_dot_product_attention(q, k, v, attn_mask=mask)
    x = rearrange(x, "B H L D -> B L (H D)")
    return x
```